

University of New York Tirana

Special Topics in CSCI (Artificial Intelligence)

Course Project

Project: Bayesian Networks, Sampling, and Learning

Document type	Project brief
Level	UNYT Undergraduate
Focus	Bayesian networks, probabilistic inference, sampling, MLE, and EM
Deliverables	Written report in PDF and submission.py
Submission Platform	Submit via Turnitin: FRIDAY's Class ID 52907870; Enrollment key spring2026 THURSDAY's Class ID 52907903; Enrollment key spring2026 DEADLINE: June 20, 11:59 PM

Overview

This project is applicable for UNYT, BSc (undergraduate) students with an emphasis is on reasoning, implementation, and interpretation.

Learning goals

- Represent causal structure with Bayesian networks
- Factorize joint distributions and compute marginals
- Implement forward sampling, rejection sampling, and Gibbs sampling
- Estimate parameters with maximum likelihood
- Apply Expectation Maximization when labels are latent
- Interpret conditional probability tables in a data annotation setting

Suggested software setup

Use Python 3.12. A virtual environment is recommended. The following commands assume that the project files have already been downloaded and extracted into your project folder.

```
python --version
uv init .
uv python pin 3.12
uv venv
uv pip install -r requirements.txt
```

uv run grader.py

Students may also work with another virtual environment manager if they prefer, provided the code runs correctly with the required packages.

Project Structure

The project is divided into four parts. The first two build conceptual understanding of Bayesian networks and inference. The last two focus on implementation and parameter learning.

Part	Topic	Main objective
1	Basic Bayes	Model a simple causal scenario and reason about independence
2	Sampling	Build a Bayesian network for a mutation tree and perform approximate inference
3	Learning	Estimate annotator reliability with observed labels using MLE
4	Learning without labels	Recover latent labels and parameters using EM

Part 1: Basic Bayes

Consider a simple campus scenario. On your way to class, you notice that the grass in the central courtyard is wet. A plausible explanation is that it rained, or that the sprinklers were on. Model this situation using the random variables Rain, Sprinkler, and WetGrass.

- List the directed edges in the Bayesian network that expresses the causal structure.
- Write the joint probability $P(\text{Rain}, \text{Sprinkler}, \text{WetGrass})$ in terms of local conditional probabilities.
- Write $P(\text{WetGrass})$ by marginalizing over the latent causes.
- Assume $p(\text{Rain}) = 0.2$, $p(\text{Sprinkler}) = 0.5$, and use the conditional table below to compute the probability that it rained, the sprinklers were on, and the grass was not wet.
- Mark the following as true or false: Rain and Sprinkler are independent. Rain and Sprinkler are conditionally independent given WetGrass. $P(\text{Rain} \mid \text{WetGrass}) > P(\text{Rain} \mid \text{WetGrass}, \text{Sprinkler})$.

Rain	Sprinkler	$P(\text{WetGrass} \mid \text{Rain}, \text{Sprinkler})$
True	True	0.9
True	False	0.8
False	True	0.7
False	False	0.0

Part 2: Sampling

You will model the inheritance of a short DNA sequence across a small phylogenetic tree. Each node stores a nucleotide at each sequence position. A descendant mutates from its parent with probability

mutation_rate. When mutation occurs, the nucleotide changes uniformly to one of the other three nucleotides.

Assume the four-node tree below is already specified in the starter files by name. Your task is to encode it as a Bayesian network and use it for sampling and inference.

- Implement `initialize_phylogenetic_tree(mutation_rate)` in `submission.py`. Define the nodes, domains, parent structure, and conditional probability tables.
- Implement `forward_sampling` to sample one full observation from the network in topological order.
- Implement `compute_joint_probability` for a complete assignment.
- Run one forward sample with `mutation_rate = 0.1` and `genome_length = 10`, and record the sampled genome for each species.
- Implement `rejection_sampling` to estimate $P(\text{ThomasBayus} = \text{AAAC} \mid \text{KeniusBayus} = \text{AAAA})$.
- Implement the iteration loop of `gibbs_sampling` for the same model.
- Compare rejection sampling and Gibbs sampling using 100 and 10,000 steps, then report both estimates next to the exact posterior.

Method	Steps / iterations	Estimated posterior
Rejection sampling	100	
Rejection sampling	10,000	
Gibbs sampling	100	
Gibbs sampling	10,000	
Exact	-	

Part 3: Learning from Observed Labels

Now consider a quality-assessment problem in which three annotators label a conversation between a human and an LLM as good or bad. The true label is Y , and the annotators produce noisy observations A_0 , A_1 , and A_2 conditioned on Y . In this part, assume the ground-truth labels are observed.

- Draw or describe the Bayesian network structure using Y and the annotator variables.
- Write the conditional probability table of a perfect annotator.
- Write the conditional probability table of an annotator who guesses uniformly at random.
- Implement `bayesian_network_for_annotators(num_annotators, dataset_size)`. Use the shared domain [good, bad] and initialize CPTs with a prior that annotators are better than chance.
- Implement `mle_estimate` with Laplace smoothing through `lambda_param`.
- Use `annotations.csv`, build the network, and implement `mle_estimate_for_annotators` to learn the model parameters.
- Plot the CPT for each annotator and identify which annotator is least trustworthy, based on the learned probabilities.

Y	$P(A = \text{good} \mid Y)$	$P(A = \text{bad} \mid Y)$
good		

bad		
-----	--	--

Part 4: Learning without Labels

In many real settings, the true labels are not observed. You must estimate both the annotator reliability and the hidden true label distribution. In this part, implement the Expectation Maximization algorithm for the same annotator model.

- Implement `e_step(network, data)`, which creates a weighted dataset of fully observed assignments and normalized responsibilities $q(h)$.
- Implement `m_step(network, data)`, which updates the network parameters from the weighted dataset.
- Implement `em_learn(network, data, num_iterations)`.
- Run the provided evaluation routine for 100 iterations and plot the annotator CPTs and the label prior.
- Based on the plots, answer whether EM recovered the true labels and explain the evidence that supports your conclusion.

Submission Requirements

- Submit one written report in PDF format. Clearly label every answer by part and subpart.
- Submit `submission.py` for the programming section.
- Include requested tables, calculations, plots, and short interpretations where asked.
- Run the local grader before submission and verify that your code executes without crashing.

Academic Integrity

You may discuss ideas and high-level approaches with classmates, but all written answers, code, and analysis you submit must be your own. Cite any external source you rely on for explanations or background material.

All the supporting files are available [here](#)